

Lec 11 Training objective: Reconstruction Loss (MSE, Binary Cross-Entropy)

Lec 11 Training objective: Reconstruction Loss (MSE, Binary Cross-Entropy)

Lecture	11 / 63 + intro
Duration	39:03
Captions	English (auto-generated)
Words	6,106
Video	FyL6UTURU4c
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone. Welcome to the lecture series so on fundamentals of generative AI and the large language models. We have started with the week two and in the previous session we had seen the theoretical concept of autoencoders. In this particular session, we will see the math behind autoencoders, how exactly the autoencoder works from a mathematical perspective and towards the end we will derive the loss function. So the loss function here we will be looking into both In case if you are working on a tabulated data, then we will be using a mean square error MSE.

[00:36] Suppose if you are working on a data which is having the features in terms of zeros and ones. Even in uh MSE, if at all if you have a features which are continuous data, then we'll be using a loss function mean square error. If at all if you have a features which are in terms of binary, which is zero and one, we'll be using a binary cross entropy. So we'll be deriving even the loss function also. So let's get started from the mathematical point of view. So from this architecture diagram, what did we understand? So we have an input layer, we will have an encoder architecture. Then

we will have an you know latent code which we are going to obtain. From this latent code is passed as an input to to an decoder and

[01:23] decoder is going to give us the output back. For understanding, I'm considering a simple autoencoder which is having just one hidden layer. Which means the input is passed to an hidden layer. The output of that hidden layer itself is a latent representation. This hidden layer output is a passed to an decoder which is having just one layer. That is your output layer. » [snorts] » Now, if you look into this particular diagram, what are we understanding? So we have an input. So this input is having a totally six features. Okay? And then we have an hidden representation or the latent code which is having a three features.

[02:08] So, now since it is an fully connected neural network, one neuron is connected to all of the three. Second input is connected to all of the three neurons. The third feature is connected to all of the three and so on so forth. So, how do you write mathematically H? Can we write the H as the you know, input X multiplied with the weight? Correct? This is is this not the formula? And definitely this entire is a passed to some activation function. Let us assume that activation function is G.

[02:44] Okay? The activation function used at a hidden layer. Of course, we have a bias at term also, but for understanding we are not considering a bias at term. So, is it not H is given by a mathematically X that is input multiplied by the weight? And here the weight we are because it is at an encoder site, the weight is denoted as WE. WE is encoder weight. Now, how many such weights are required? Suppose if we have a thousand samples and each sample is having a six features. Let us assume this is what is the data that we have input data. Now, is it not we are going to obtain thousand samples here as well? But those thousand samples will be of dimension three features.

[03:38] So, here the input will be six such features. And what we are obtaining will be a data which is having a three. But is it not that will have a thousand such a samples? Here also you will have thousand such a samples. I hope you understood what I'm trying to explain here. » [snorts] » So, if H if [laughter] X input is having a dimension thousand into cross six. What is thousand cross six? Thousand samples, six of features. Then, we know that the latent representation dimension is three. Then, is it not at the H we are going to have this thousand cross three?

[04:25] Yes? If that is the case, what should be our weight matrix? What should be the dimension of the weight matrix? How many such weights are required? How many weights will be here? Can you see from this mathematical one? Can you able to tell? If one is thousand cross six, we know the output is thousand cross three, then what is that you should have here so that you get the output thousand cross three? Is it not six cross three?

[04:57] So, the weight matrix will be six cross three. Which means you will be having

a six such you know, six cross three will be the dimension of this particular weight matrix. Now, I hope this is clear to you. It's very simple. Ultimately, our aim is to get that thousand you know, is to get this six features back. If you have to get that a six features back, mathematically how it is happening is what we are trying to understand. So, this H is obtained from X multiplied by the weight.

[05:34] And if that is the case, X dimension is a thousand such a samples are there with the six features. And latent representation is a thousand such a samples but three features because the dimension should be smaller than the input dimension. If that is the case, what is the weight matrix that we are considering? The weight matrix is a 6 cross 3 because this and this will be same. Then you're going to get this output. That is what is 1000 cross 3 we have obtained. So what is the weight matrix W_E role here?

[06:08] Is it not a W_E is mapping an input vector which is having a dimension R_6 to a dimension R_3 ? That's what is the role of W_E or the role of weight matrix W_E . Okay, if that is the case, let's write mathematically what is this equation $\hat{X}$. $\hat{X}$ is obtained from H thus multiplied with the weight W_D . What is W_D here? D is the weight matrix at a decoder side. Correct? Now we know what is this size. This size is 1000 cross 3, right? And what is our ultimate aim? Ultimate aim is to get that six features back. Now what should be this value so that you get 1000 cross 6? Is it not 3 cross 6?

[07:02] Right? That's what has been explained here. So $\hat{X}$ is obtained from X H multiplied with the weight matrix. What weight? It is the weight at a decoder side. And of course there are some activation functions also. The activation function at an hidden layer is denoted as a G and the activation function at an output layer is denoted as F . Let us understand what activation functions in a while. So is it not this? If that is the case, what is the weight matrix here W_D ? How many such weights you have to consider here?

[07:39] If your ultimate output is 1000 cross 6 And if your input is 1,000 cross 3, what should be the weight matrix at a decoder? Is it not 3 cross 6? So, what is the W_D is doing here? W_D is a weight matrix, which is a mapping the latent representation, which is in R_3 to R_6 back. Yes? Okay. So, can you observe something here? Uh one is a W_E and the other one is W_D . Can you see here? One is of size 6 cross 3 and other one is of size of 3 cross 6. What are we understanding? Uh is it Can you say it's a transpose? Yes? But not necessarily that the values should be same, right? Here, the uh shape is, uh you know, reversed, but the values can be anything. Do you agree to this? One is having a 6 cross, you know, 6

[08:38] um rows and then three columns. That is the size of one matrix. The other one is uh three rows, but we have six columns. We have six columns. So, but the value inside can be anything here, right? So, only the shape is reversed, but the values can

be anything. Uh if if you agree to this particular if you're convinced with this, what exactly is the problem? Are you convinced with whatever has been explained so far? Yes? So, one is of size 6 cross 3. This particular matrix is of size 3 cross 6. Right? But the values inside this matrix and the values which are there inside this matrix are different. If that is the case, is it not that you have to learn the two different weights? Uh during your back

[09:28] propagation, we have to learn two different weights here. The model has to learn the WE and the model has to learn WD also. So, to in order to achieve both encoding and decoding a process, both the WE and WD are to be learned. So, [snorts] is it not a time-consuming one? Is it not more number of parameters we have to learn? Because all these weights has to be learned and all these weights has to be learned. If this is your approach, this is called a generic setting. What is a generic setting? This is important concept.

[09:59] Generic setting is to in order to achieve encoding and decoding a process, you have to learn both WE as well as WD. If that is the case, then it is a time-consuming a process. Training a time will be more. Uh rather, what can be an alternative solution? Can we think of this? Can we learn only one weight matrix, that is the WE? And other one, can we take a transpose of it? Because anyhow, the shapes are different, right? I mean, just the shapes are reversed. Can we learn only one weight? For the other side, can we take the transpose of the same weights to achieve the decoder decoding at the decoder for reconstruction? Can we do that? Answer is yes. If that is the case, it is called a tied state as a

[10:46] weight assumption. Tied weight assumption. What is a tied weight assumption? Just learn only one weight. For the other one, take the transpose of it. Why? Can you see? W of E is 6 cross 3. What is the transpose of it? Is it not 3 cross 6 is the transpose? Is it not that is the the dimension that is required for a decoding a process? Yes. So, with this, are we not reducing the complexity of the model? Uh the model need not have to learn two different weights. The model has to learn just one uh set of weights.

[11:20] For the other one, just take the transpose of it. If that is the case, you will be calling this as an uh tied weight assumption. So, what have you understood here? Uh H is obtained by multiplying input with the weight and X cap is obtained. That is a reconstruction is obtained by multiplying H with the weight. Now, at both the sides there is an activation function. At hidden layer, we are denoting it as a G. At an output layer, we are denoting it as F.

[11:50] And if you look into the shape, only the shape is reversed, but the values can be anything. So, to achieve encoding and decoding, if you are learning the weights, we call it as a generic setting. But this is a time consuming. In order to avoid, because anyhow the shape is reversed, let's learn one weight for the other one take the

transpose. If that is the case, it is called a tied weight assumption. Now, let us understand what are those activation functions. So, if at all, if you are aim is you know, reconstruct ultimately the aim is to reconstruct the data. If if the input features are binary, what is binary? If all the features we have a data set, all the features here are in terms of zeros and ones. I'm

[12:42] talking about the features. These are the features X_1 , X_2 , X_3 . These are the features. If all of these features are in terms of zeros and ones, then ultimately if you consider this as an input, ultimately it has to reconstruct the data in terms of zeros and ones only. So, if your input data, if the input features are binary, then the aim is the output decoder also should give the data which should lie between zero and one only. Right? It has to be zero and one. So, if that is the case, what should be the activation function that we'll be using at an output layer? It has to be sigmoid. Why sigmoid? Because it will arrange the values between zero and one. Any value which is greater than 0.5 will be one.

[13:32] Any value which is less than 0.5 will be zero. So, is it now you obtained the values in terms of zeros and ones? That is what is a sigmoid. What is a sigmoid here? So, the role of a sigmoid here is where is that we'll be using? We'll be using this at an output layer. So, is it not $\hat{X}$? $\hat{X}$ is a reconstructed one. That is obtained by passing latent representation multiplying with the weight and ultimately you have to pass it to two and activation function. That activation F here is in a sigmoid. What is a sigmoid? $1 / 1 + e$ to the power of minus whatever is that a term. If that term is denoted as a , it is minus a .

[14:16] Okay. This is what at the output layer when the features are binary. If that is the case, what should be the activation function at an hidden layer? What is hidden layer here? That latent representation. Is it not H is a latent representation? How do you obtain this? Is it not X getting multiplied with the weight and it is passed to an activation function and that activation function here is G . We are denoting it as a G .

[14:42] And that G can be either a ReLU activation or it can be an sigmoid activation. So, at an encoder at the encoder site, you will be using an ReLU or sigmoid as an activation function. At an output layer, you'll be using a sigmoid if you're using an binary features. Now, what if we have an data set but those data set are continuous values. Now, for example, you have an data set. Let us assume this is the data set that we have.

[15:20] And here we have 2.5, 7.3, 6.9, and 11.4. You have a data set something like this. What is our ultimate aim? When you're reconstructing, you should get to the values something similar to to 2.1. You should get 6.4, then you should get 6.9. Model will not to predict correctly 6.1, then 10.7. So, this is what your ultimate aim is to reconstruct once again the continuous values only. If that is the case, what should be the activation function at the output layer?

[15:54] It has to be linear. So, $\hat{X}$ is an output. So, it is obtained by multiplying H with the weight and passed to an activation function, and that activation function is linear activation. And what is linear? $F(A) = A$. Whatever you pass it to that function, you should get the same output. So, $F(A) = A$. Now, if that is the case, what should be the activation function at the hidden layer or at the encoder part, the encoder is a hedge.

[16:25] It gives us the hidden representation, latent representation hedge. How do you obtain X multiplied by the weight? So, that you're passing it to an activation function. That activation function can be either ReLU or it can be sigmoid or it can be tan hedge. That mainly depends on what is the architecture we are what model we are using it. So, what are we understanding from this particular slide? So, in the previous one, we have seen that there is an activation function.

[17:01] So, we have F and G . So, what are those activation functions? On what basis will be choosing those activation functions? That purely depends on what is the data that you're trying to reconstruct. If you're trying to reconstruct the binary, which means your input should be binary. The features are binary. Then use a sigmoid at the output layer. If you are reconstructing the real valued, which means a continuous data, then your the activation function should be linear at an output layer.

[17:33] At the hidden layer, usually we go for an at the latent representation, we usually go for a non-linear activation functions. So, it can be ReLU or it can be an sigmoid or it can be a tanh based on what kind of or the family of a ReLU based on what model that we are choosing, you can choose any one of these activation functions at the hidden layer. What is hidden layer? At the encoder part. So, now with this, let us try to see what is the reconstruction loss in the autoencoders. More specifically, let us see a mean squared error. Now, when do you go for a mean squared error?

[18:15] When our aim is when our input is a real valued, so we have an input which are a real valued. What is our ultimate aim? Aim is to reconstruct this a real valued. So, from the original input X , we have to get the reconstructed one which we are denoting it as $\hat{X}$ or $\hat{X}$. What is our ultimate aim? That output should be exactly similar to an input. Suppose if the input is a 2.5 7.5 8.5. Our ultimate aim is also getting 2.5 7.5 8.5, which means exactly if this is the input X , the reconstructed one $\hat{X}$ or $\hat{X}$, then our ideal requirement is both should be exactly similar. Our ideal requirement is $\hat{X}$ should be equivalent to or should be equal to X , but in practical that's not possible, right? So, our aim

[19:15] is at least approximately let it be equal. So, this is what is the requirement. Now, what is our you know aim? Aim is to minimize the difference. Difference between what? Between the original input and then the reconstructed input. You see, we are talking about autoencoder, which is a special category of unsupervised learning. In unsupervised learning, we will not have the labels. Then what is the reference

to the model? The reference to the model here is autoencoders are mainly responsible for reconstruction. If it is a reconstruction, reconstructing what?

[19:53] Original input. Can we consider original input only as a reference when reconstructed data is obtained to calculate the loss? Yes. Therefore, the difference is original input x and reconstructed $\hat{x}$. This is what the differences should be minimized. Per sample loss, if there is just one sample, our loss function will be particular that particular sample original input and its a reconstructed one. The loss can go for a negative value, therefore we are taking a square of it.

[20:29] So, we are taking a square of it so that the loss is at all if there is a negative, we are going to convert that into a positive. This is per sample loss. But it is not one sample, it is over all samples. If it is over all the samples, then this is a per sample, which is denoted as l_i , and a per over all m samples, it is denoted as you are taking an average of it. So, if at all if you have n samples, $1 / m$ summation of all of those values.

[21:00] This is what is the total average. Now, what is our ultimate aim? Is it not our ultimate aim is this that you know, get our ultimate aim is to minimize the difference between the original input X and then the X reconstructed one. But, how are we obtaining this? What is playing an important role here? Of course, the weights. So, how is that you will be get that W which will help us to minimize the loss between X and $\hat{X}$? Correct? We need that W which can help us to minimize this difference between original input and the reconstructed one. Okay.

[21:45] If this is the case, this is our ultimate aim. Aim is the model has to learn that W because if it learns just one W , the other side is transpose. So, if you can just get that one W which can help us to minimize this difference, that is our ultimate objective. Okay. Now, from our previous slide, we have already understood what is weight here. So, the weights here are W_E and W_D . Encoder and decoder, but we have already seen that one weight is sufficient. So, can I assume W_E as just W just for notation sake?

[22:25] Can we write it as W ? And W_D will be the transpose of W only. So, yes, this is what we we have seen it from our previous one. Okay. Now, from the previous, we have also seen something this. What is $\hat{X}$? The $\hat{X}$ can be written it as a hatch multiplied by W_D . Correct? That is what is $\hat{X}$. Is hatch multiplied by W_D . Can we substitute that into this? And just from this uh we know that it is W transpose. So, can I substitute W_D as a W transpose? Why?

[23:09] Because that's our assumption, right? And what is our H ? Uh from our previous slides, we know what is a H . H is input that is getting multiplied by the weight matrix W_E , right? So, this H can I substitute? So, since the H is X multiplied by W_E , right? And now we know that W_E is just W for a notation's sake. Can we see this? So, is it not uh This is not very complex. It's a simple math. Ultimately, our aim is we

now need a weight matrix W which will help us to minimize the difference between X and reconstructed one.

[23:56] Just to substitute the values. What are the values that you're substituting? Just substitute the equations. We know $X = H W^T$. Substitute that. And the W^T , we know that it is W transpose. So, take the W transpose. And then we know what is H . H is $X W$. So, under W , you substitute it as a W . What have we obtained? Our aim is to get that reconstruction loss. So, this is what is the reconstruction error.

[24:27] Reconstruction error is the model should learn that optimized W such that it will differentiate it will give that difference between original input and reconstructed data less. Okay? That is what is minimum is this difference should be minimum. Right? And how it is possible? It is possible with the help of W . Okay. Now, uh so, we are understanding that W is playing a role here. Uh, so, now, large value of W can reduce this difference, or even the smaller value of W also can bring out this difference, which means what I'm trying to say, if your aim is a 3 cross 6 to learn the weight matrix of 3 cross 6, which means there are three rows.

[25:18] These are the three rows. Now, there are six such. One, six. If your aim is to learn this, the values inside can be anything. You can have a larger value, you can have a smaller value. Uh, it can be any values, right? So, we are now introducing an regularization term. What is regularization? Ultimately, aim is to minimize the difference between this with the help of this weight matrix W . So, our aim is get the this W a smaller value.

[25:58] So, whatever is this values which are there inside, those values should be smaller. How you will get those values so smaller? Can I take a lambda some smaller value? Can I take lambda as 0.01? Now, can I multiply 0.01 with this entire matrix values? Can we do that multiplication? Then, whatever may be the values which are inside this, when you multiply that with a smaller value, is it not the values now are smaller?

[26:30] What are we achieving from this? What we are trying to do? Our aim here is to minimize the loss. What is the loss? The difference between original input and reconstructed. With the help of what? With the help of weight. So, anyhow, you are getting a weight matrix. Let the values inside that matrix be smaller is what we are ensuring by adding a this regularization term. So, this is what the value in the W matrix can be small or large, but always ensure W takes a smaller value.

[27:01] By how are you ensuring W takes a smaller value? This is what is the regularization term. Therefore, we are using this regularization term. So, now once you complete the entire training, you have completed the training. So, towards the end of the training, what are we achieving? Are we not achieving the optimized weight matrix W^* ? We are getting the optimized weight, right? So, what is the role of a

training? Role of a training is to reduce the training error. How is that it will reduce? It will compare with the original input and reconstructed input. So, it will reduce the difference between these two. So, when you are trying to reduce during back propagation, is it not you'll be constantly updating those weights also?

[27:48] So, towards the end of the training, you are going to get the optimized weights. Okay, if you have obtained the optimized weight, what is that we what is that we get? So, what is a H now? H is input getting multiplied with this optimized weight. When you multiply with this optimized weight, what is that H we get? We get the optimized H . Correct? That means the most important features are now present here. Now, how do you get this $\hat{X}$? $\hat{X}$ is this H , which is the most optimized one, when that is getting multiplied with the optimized weight transpose.

[28:30] What we are going to obtain? Is it not we are going to obtain the reconstructed data, which is as similar to the original one? Why? Because you have obtained optimized weights, which which can reduce that difference. If you can reduce that difference, then definitely you are now obtaining that optimized H star. So, what is this This latent representation is very good. Now, that latent representation, when you multiply it with the transpose of those optimized weight, we are going to get this $\hat{X}$. Now, this $\hat{X}$ or $\hat{X}$ definitely, when you now compare it with the original, this will be definitely closer to the original one. The reconstructed output $\hat{X}$ is now a close approximation to the original.

[29:22] Right? So, is it not we have achieved you know, reconstructed data? This is the mathematical explanation of what we have seen in the previous session. Theoretically, we had seen how H is obtained. From H , how reconstructed the data. But here, we are seeing mathematically and we have also derived the reconstruction loss. This is what is with respect to mean square error. What if you are dealing with the binary features? So, can we still use a mean square error only? Answer is no, because here the features are in terms of zeros and ones. So, binary data, the features are in terms of zero and one. If that is the case, what will be the activation function? Definitely, sigmoid will be the activation function. Okay.

[30:11] Now, let us see what is the loss function here. So, per sample, what is the per sample error for a mean square? It is just X minus $\hat{X}$ square. You are taking a difference between X and $\hat{X}$. And to normalize, you are taking I mean, to avoid a negative, you are taking it as a square. But here, the features are in terms of binary, which is in terms of zeros and ones. Then, if that is the case, what will will the loss function? The loss function here is the original you see here the original features will be in terms of zeros and ones and ultimately what is that the model will predict sigmoid will give the value between zero and one the probability. So here the original value is one but the model

[31:03] has a predicted the probability of being it one as 0.92. The original is a zero

the model has a predicted this probability. Original is one the model has predicted this. Original is one the model has predicted zero and the model has predicted 0.12. » [snorts] » So when you are now taking calculating the error you cannot use the mean square error. It is in binary cross entropy. What is binary cross entropy? This is the formula for binary cross entropy.

[31:32] Right? Now here you can observe there is a negative term. Why negative term here? What is the role of a negative term? There in order to obtain a positive value you have squared it. But here we cannot square it right? So what is the way that you are making the error term as a positive by my by writing by multiplying it as minus? Why to multiply it as minus? What is that then this term is going to give us? Definitely this term will give us negative right? Because this term gives us negative you are multiplying it as multiplying with the negative so that the term becomes positive. Now why this term gives negative? Ultimately what is this a predicted value? It's a probability. And probability values will

[32:21] always between zero to one. This the probability value will be between zero and one. If that is the case you have a log term. So for a log values if the values are between zero and one definitely the log value will be negative, right? Which means this entire inner expression, the value will be negative. Uh so, if the value is negative, to make it as a positive, you have to multiply it with a negative term. So, we put minus sign. I hope you're understanding this. What What we are seeing is a reconstruction loss. And what is reconstruction loss? We have already seen sometime back reconstruction loss. But that we have seen with respect to if at all if you have a features which are real valued.

[33:09] But what if the features are binary? If the features are binary, then you cannot use mean square error. It has to be in binary cross entropy loss. Then what is binary cross entropy? It is This is the binary cross entropy. The original input and then the predicted value. Now, since and there is a negative term. Why negative? Because ultimately we want loss to be positive. So, if loss has to be positive, why to multiply it with the negative? Because this whole term is giving us the negative value. Why this term gives us the negative value? Because this is the probability and the probability will always between zero to one between zero and one. So, for any value strictly between zero and one, the

[33:54] log value will always be negative. Right? So, now if it is a negative, the inner expression is giving us the negative value. To make it as positive, it is a minus sign. This is a per sample. But it is for all the samples, what is that you will do? See, here this is you have a D number of a features. So, what we are now representing, I represents the sample. If I is one, so this is X11, X12, X13 and so on up to X1 D. So, which means it is for the first sample.

[34:32] For [snorts] one sample, this is the loss function. What if you have over n

number of samples, then if it is a for n number of samples, it is $1/n$, where I value is moving from 1 to n . So, I represents the sample number. So, if there are n number of such a samples, it is $1/n$. Then, what is the total loss? The total loss is minus this minus term. And in this, you substitute the values. This is the loss function that we will be using for an auto in the auto encoder if you are using binary features as an input. So, what is ultimate aim? This loss between X and $\hat{X}$ should be minimized and get that optimized W . And here also, the aim is get that W where W value will be smaller, therefore the

[35:36] regularization term. So, what are we understanding from this? In case, if we are using features as real valued, then go for mean square error loss and reconstruction loss here. Why reconstruction loss? Because you are comparing with the original input X only. And if you are using features as binary, then use binary cross entropy and this is the loss function for binary cross entropy. With this, we have come to an end of this particular session. And in this session, we have seen the [snorts] loss function for both mean square, you know, real valued as well as binary features. So, let's have a quick revision. So, there is an input that is a passed uh, uh, an uh, Encoder is going to give us the latent

[36:29] representation. And here the latent representation is an hyper parameter, which we have to fix what should be the size. So here we have fixed it as two. Therefore, you will have in uh, such a features, but in such a samples, but a two features. That is passed as an input to a decoder, and a decoder has to reconstruct the, uh, data back. So, um, since it is an a tabulated data, this is tabulated data. So therefore, this architecture, encoder and a decoder should be a multi-layer perceptron. Can be implemented as multi-layer perceptron. What is the aim? X when it is passed to an encoder, you get this, uh, Z . Either take it as Z , or if you want a common notation, similar notation, take it as H . That H when it

[37:14] is passed to a decoder, you get X dash or $\hat{X}$. Right? Then, what is the loss function we are using? The loss function is, uh, the mean squared, uh, loss. Uh, aim is to, uh, get the difference between X and X dash or $\hat{X}$. This is with respect to tabulated data. But what if you want to reconstruct an image? Then, uh, what will be the, uh, encoder and decoder architecture? The encoder and decoder architecture will be an convolutional, uh, layers. For an image, the encoder uses an convolutional layer to extract the, uh, features. And a decoder uses an uh, upsampling or a transpose convolution. In short, it is a CNN based. So, uh, for this kind of an application where, see, either you can have an tabulated data, or you can have

[38:06] a data with respect to images. So we are not dealing with respect to audio and a video here. We are, uh, dealing with the only tabulated data and images. If you are, uh, dealing with the tabulated data, go for multi-layer perceptron architecture at encoder and decoder. If you are going for an uh, image based, then use an CNN based auto encoder. So, with this we have come to an end of this session. In the next

session, we will start from this point only. What if there are images? So, we have seen if it is a tabulated data.

[38:40] What if there are images? How do it works? So, we will first see a CNN based auto encoder. Then we will see two simple auto encoders that is a simple shallow auto encoder and a deep auto encoder. With this, we have come to an end of this particular session. Thank you.